

SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA11

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models. (BL4)

Assessment Tool Weightage: 40%

Code of Conduct:

*I **D Varshini / 192320007** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Case study

QUESTIONS: Any 4

Total Marks:40

1. Online Hotel Reservation System

A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

Question:

Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

ANSWER :

1. System Analysis

The Online Hotel Reservation System allows customers to search hotels, book rooms, make online payments, and submit reviews. Hotel managers can manage room availability and pricing, while administrators oversee users, bookings, and transactions.

Main Objects (Classes)

- User
- Customer
- Hotel Manager
- Administrator
- Hotel
- Room
- Booking
- Payment
- Review

2. Application of Object-Oriented Principles

a) Encapsulation

Encapsulation means hiding data and allowing access through methods.

Example:

- The Booking class keeps booking details (booking ID, customer name, room number, booking status) as private attributes.

- Access is provided using methods such as:
 - createBooking()
 - cancelBooking()
 - getBookingDetails()

This protects booking information from unauthorized modification.

b) Inheritance

Inheritance allows one class to inherit properties and methods from another.

Example:

User (Parent Class)

- UserID
- Name
- Email
- Login()

Child Classes

- Customer
 - Search hotels
 - Book rooms
 - Make payments
 - Write reviews
- Hotel Manager
 - Update room availability
 - Change room prices
- Administrator
 - Manage users
 - Manage bookings
 - Monitor transactions

This reduces code duplication and improves reusability.

c) Polymorphism

Polymorphism allows the same method to perform different functions depending on the object.

Example:

Method: displayDashboard()

- Customer: Shows hotel search, bookings, and reviews.
- Hotel Manager: Shows room management and pricing.
- Administrator: Shows user management and reports.

Another example is makePayment(), where different payment methods (Credit Card, Debit Card, Digital Wallet) implement the same method differently.

3. Suitable SDLC Model

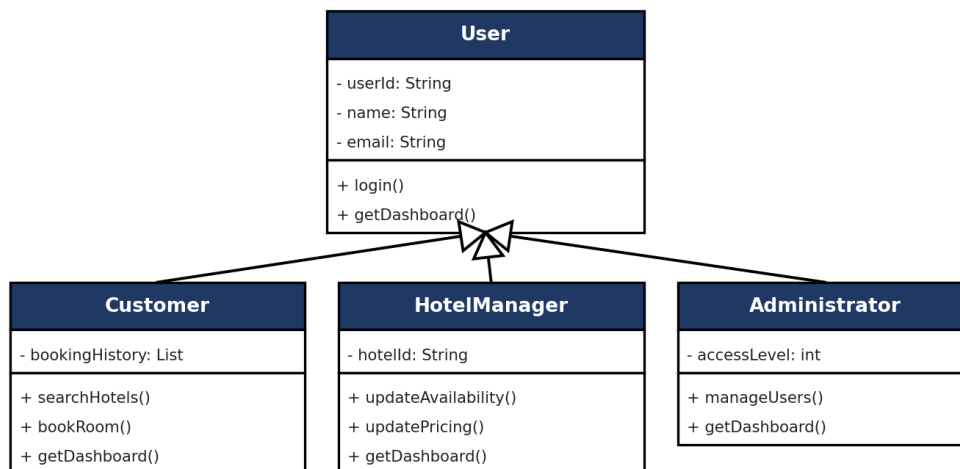
Agile SDLC Model

The Agile model is the most suitable for this system.

Justification

- Customer requirements may change frequently.
- Features such as booking, payment, reviews, and notifications can be developed in separate iterations.
- Regular feedback from customers and hotel managers helps improve the system.
- Bugs can be fixed quickly after each sprint.
- New features can be added without affecting the entire system.

Hotel Reservation System — User Inheritance Hierarchy



Each subclass overrides getDashboard() to return role-specific content — an example of polymorphism. Attributes such as bookingHistory and hotelId are kept private (encapsulated) within their own classes.

4. Conclusion

The Online Hotel Reservation System effectively applies encapsulation by protecting sensitive data, inheritance by sharing common user features among different user types, and polymorphism by allowing different behaviors for common methods. The Agile SDLC model is the best choice because it supports continuous development, flexibility, customer feedback, and frequent updates.

2. University Management System

A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system.

Question:

Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

Answer :

1. System Analysis

The University Management System automates major university activities such as **student admission, course registration, examination management, and result processing**. Different users interact with the system based on their roles.

Main Users

- **Student** – applies for admission, registers for courses, views exam schedules and results.
- **Faculty Member** – manages courses, attendance, marks, and examination activities.
- **Examination Staff** – schedules examinations and processes results.
- **Administrator** – manages students, faculty, courses, and system operations.

2. Key Classes

The following classes can be identified:

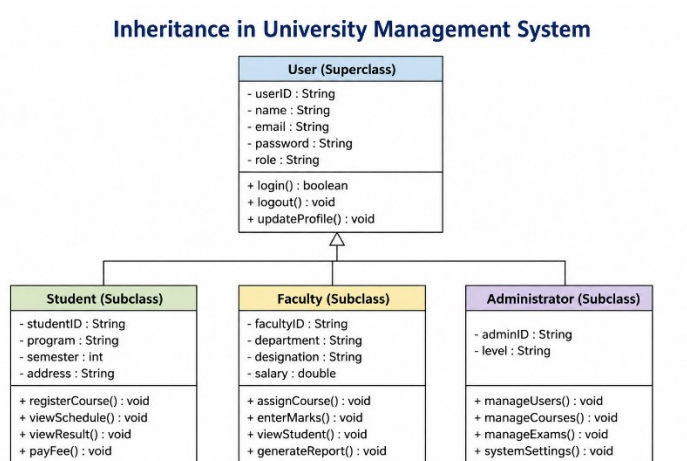
Class	Main Attributes/Functions
User	userID, name, email, login()
Student	studentID, program, registerCourse(), viewResult()
Faculty	facultyID, department, teachCourse(), enterMarks()
Administrator	manageUsers(), manageCourses()
Course	courseID, courseName, credits, courseDetails()
Registration	registrationID, date, registerCourse()
Examination	examID, date, venue, scheduleExam()
Result	marks, grade, calculateGrade()
Department	departmentID, departmentName

Admission	applicationID, status, processAdmission()
------------------	---

3. Object-Oriented Relationships

a) Inheritance

A common **User** superclass can be created.



- Student inherits common properties such as userID, name, and email.
- Faculty inherits the same common user properties.
- Administrator also inherits common user functionality.

This provides **code reusability** and avoids duplication.

b) Association

A **Student** can register for multiple **Courses**, and a course can have multiple students.

Student ----- Registration ----- Course

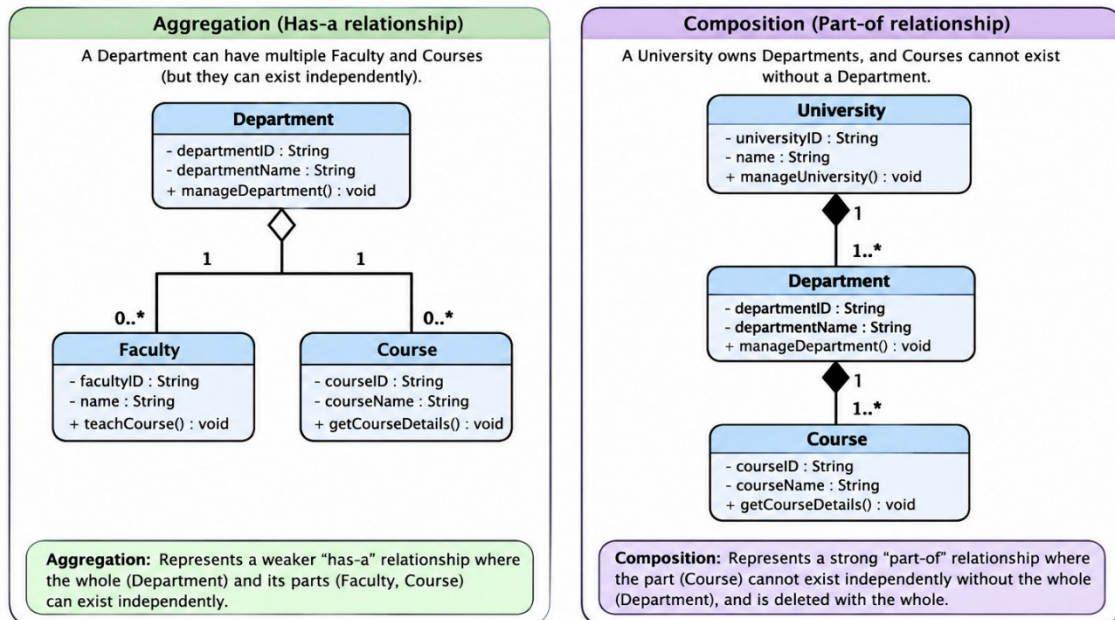
Similarly:

Faculty ----- teaches ----- Course

A faculty member can teach one or more courses.

c) Aggregation/Composition

A **Department** can contain multiple courses and faculty members.



4. Encapsulation

Encapsulation protects the internal data of objects by keeping attributes private and providing methods to access or modify them.

Example:

The Student class may contain:

private studentID

private marks

private courseList

Methods such as:

registerCourse()

viewResult()

updateProfile()

can be used to access or modify the data.

This prevents unauthorized changes to sensitive information such as **student marks and examination results**.

5. Polymorphism

Polymorphism allows the same method to behave differently for different objects.

For example, a method:

displayDashboard()

can behave differently:

- **Student:** displays registered courses and results.
- **Faculty:** displays assigned courses and student marks.
- **Administrator:** displays user and university management options.

Similarly, generateReport() can generate different reports depending on the user's role.

6. Recommended SDLC Model – Agile Model

The **Agile SDLC model** is suitable for developing the University Management System.

Reasons:

1. **Changing requirements:** University requirements may change during development.
2. **Incremental development:** Modules such as admission, course registration, examination, and results can be developed separately.
3. **Continuous feedback:** Students, faculty, and administrators can provide feedback after each iteration.
4. **Early testing:** Each module can be tested and corrected before integrating the complete system.
5. **Easy maintenance:** New features such as online fee payment or attendance tracking can be added later.
6. **Reduced risk:** Problems can be identified early rather than at the end of development.

7. Conclusion

The University Management System can be effectively modeled using **object-oriented concepts**. Classes such as Student, Faculty, Course, Examination, and Result represent the major entities. **Inheritance** provides reusability, **encapsulation** protects sensitive data, **polymorphism** supports different user behaviors, and **associations** represent relationships between objects. The **Agile SDLC model** is recommended because it supports incremental development, continuous feedback, testing, and changing university requirements.

3. Online Retail Shopping Platform

A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

Question:

Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

ANSWER :

1. Modular Software Design Using Object-Oriented Techniques

Modular software design divides the e-commerce system into smaller, independent modules. Object-oriented programming (OOP) helps achieve this using **classes, objects, encapsulation, inheritance, abstraction, and polymorphism**.

Main modules can be:

- **Customer Module:** Registration, login, product browsing, cart, and order placement.
- **Product Module:** Product details, categories, prices, and availability.
- **Vendor Module:** Vendors can add, update, and remove products from their catalogs.
- **Order Module:** Creates and manages customer orders.
- **Payment Module:** Handles online payment and payment verification.
- **Delivery Module:** Tracks shipment and delivery status.
- **Inventory Module:** Maintains stock levels and updates inventory after orders.
- **Admin Module:** Allows administrators to monitor transactions, users, vendors, and inventory.

OOP techniques:

- **Encapsulation:** Customer, Product, Order, and Payment data are protected inside their classes.
- **Abstraction:** Payment can be represented by a common Payment interface without exposing payment-processing details.
- **Inheritance:** Different users such as Customer, Vendor, and Admin can inherit common properties from a User class.
- **Polymorphism:** Different payment methods such as CreditCardPayment, UPIPayment, and NetBankingPayment can implement the same Payment interface.
- **Composition/Aggregation:** An Order can contain multiple OrderItem objects, while a Vendor manages multiple products.

This modular approach makes the system **reusable, maintainable, scalable, and easier to test**.

2. Evaluation of SDLC Models

SDLC Model	Suitability
Waterfall	Simple and structured, but changes are difficult and testing occurs late. Not ideal for e-commerce requirements that may change.
Spiral	Good for identifying risks, especially security and payment risks, but it is complex and costly for continuous development.
V-Model	Provides strong testing and verification, but is rigid and does not easily accommodate changing requirements.
Agile	Supports iterative development, frequent customer feedback, continuous testing, and changing requirements.
Incremental Model	Allows the platform to be developed and delivered in separate functional modules, such as products, orders, payments, and delivery.

3. Most Appropriate SDLC Model

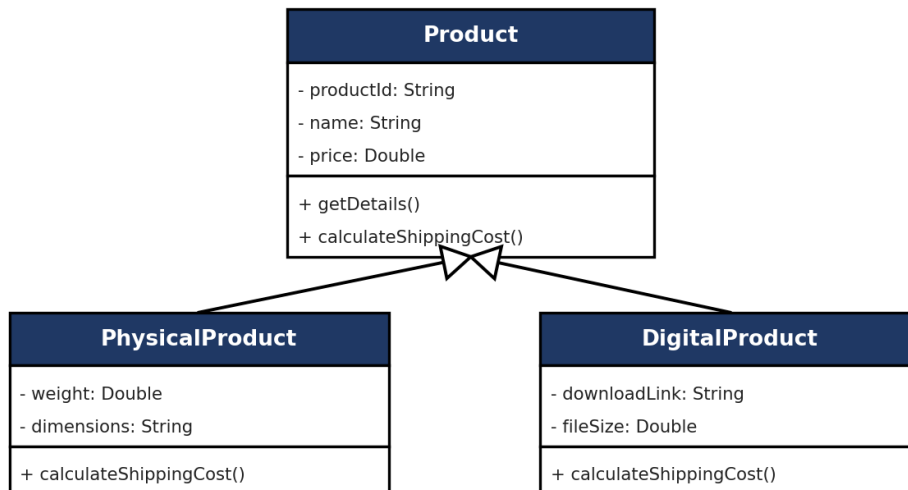
Agile SDLC is the most appropriate model for this e-commerce platform.

The system can be developed in short iterations/sprints. For example, the first sprint can develop customer registration and product browsing, followed by cart and ordering, payment, delivery tracking, and finally vendor/admin features.

Agile is suitable because:

1. E-commerce requirements can change frequently.
2. Customers and vendors can provide continuous feedback.
3. Features can be released incrementally.
4. Testing and bug fixing can be performed in every sprint.
5. Security and payment-related issues can be identified early.
6. The modular OOP design supports independent development of different features.

Retail Platform — Product Inheritance Hierarchy



calculateShippingCost() is overridden in each subclass (physical items incur courier cost; digital items return zero) — polymorphism through method overriding.

Conclusion: Combining object-oriented modular design with Agile SDLC provides a flexible, scalable, maintainable, and reliable solution for the e-commerce platform.

4. Smart Waste Management System

A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application.

Question:

Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.

ANSWER :

1. Object-Oriented System Development

The smart waste management system can be designed using object-oriented principles by dividing it into independent classes and modules.

Main classes:

- SmartBin: Stores bin ID, location, capacity, and current waste level.
- Sensor: Monitors the waste level and sends real-time data.
- CollectionVehicle: Stores vehicle details and receives collection requests.
- Citizen: Can report waste-related problems through the mobile application.
- WasteReport: Stores complaint details, location, type, and status.
- Notification: Sends alerts when a bin becomes full.
- Admin: Monitors bins, complaints, vehicles, and collection activities.

OOP principles used:

- Encapsulation: Sensor readings and bin information are protected within their respective classes.
- Abstraction: The system hides complex sensor and communication processes and provides simple operations such as `checkWasteLevel()` and `sendAlert()`.
- Inheritance: Different users such as Citizen and Admin can inherit common properties from a User class.
- Polymorphism: Different notification methods such as SMS, mobile notification, or email can implement a common notification interface.
- Composition: A SmartBin contains sensors, while a WasteReport is associated with a Citizen and a location.

This modular design makes the system easy to maintain, test, modify, and expand.

2. Suitable Life Cycle Model

The Spiral Model is best suited for this system.

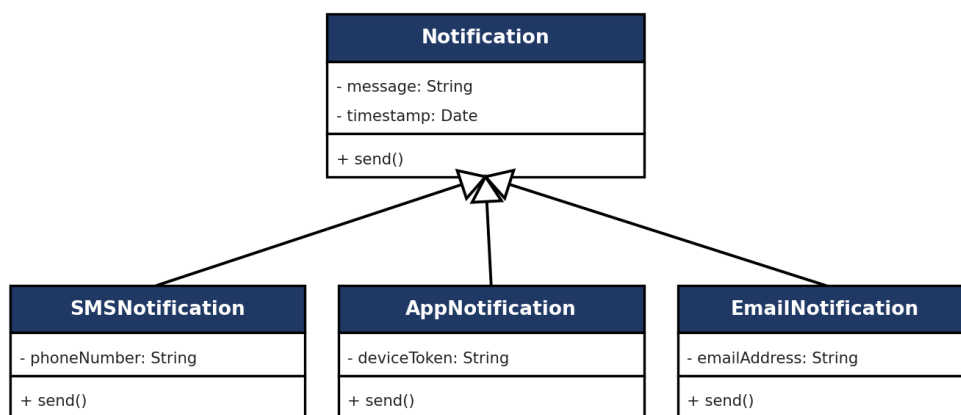
The Spiral Model combines iterative development with continuous risk analysis. This is important because the system involves IoT sensors, real-time data, communication networks, and mobile applications.

Why Spiral Model is suitable:

1. Risk management: Sensor failures, network issues, and inaccurate readings can be identified and addressed early.

2. Real-time requirements: Sensor functionality can be developed and tested through repeated iterations.
3. IoT integration: Hardware and software integration can be tested progressively.
4. Changing requirements: New features such as GPS tracking, route optimization, or additional sensors can be added in later cycles.
5. Continuous testing: Each iteration allows testing of sensor accuracy, alerts, mobile applications, and system performance.
6. User feedback: Municipality staff and citizens can provide feedback after each development cycle.

Smart Waste Management — Notification Inheritance Hierarchy



send() is overridden per channel, so BinMonitor can call notify.send() on any Notification object without knowing which channel is used underneath — polymorphism.

Conclusion

The Spiral Model with object-oriented design is appropriate because it supports iterative development, risk management, sensor integration, real-time processing, and changing requirements. It results in a reliable, scalable, and maintainable smart waste management system.

5. Smart Healthcare Management System

A multi-specialty hospital plans to develop a Smart Healthcare Management System where patients can book appointments, consult doctors, access medical records, and make online payments. Doctors update patient diagnoses and prescriptions, while hospital administrators manage departments, staff, and billing information.

Question:

Analyze the system and explain how **object-oriented principles (encapsulation, inheritance, and polymorphism)** can be applied. Also, recommend a suitable **SDLC model** for developing this healthcare system and justify your choice.

ANSWER :

1. Object-Oriented Principles

The Smart Healthcare Management System can be developed using **object-oriented programming (OOP)** by dividing the system into classes such as Patient, Doctor, Admin, Appointment, MedicalRecord, Prescription, and Payment.

Encapsulation:

Patient medical records, diagnoses, prescriptions, and billing details are kept private inside their respective classes. They can be accessed or modified only through appropriate methods. This provides **data security and protection**.

Inheritance:

A common User class can contain shared attributes such as user ID, name, email, and login details. Classes such as Patient, Doctor, and Admin can inherit these common properties and add their own specific functions.

Polymorphism:

Different payment methods such as CreditCard, UPI, and NetBanking can implement a common Payment interface with a makePayment() method. Similarly, different types of doctors can provide specialized consultation functions while using a common interface.

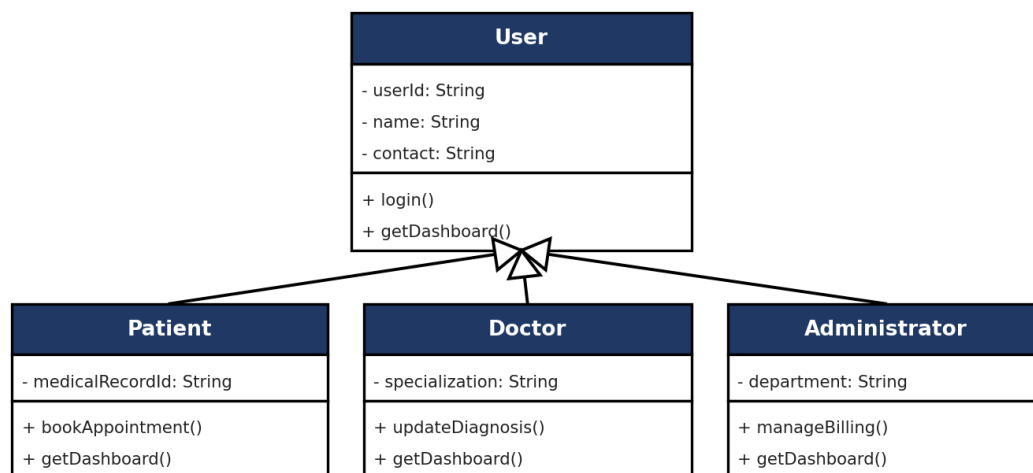
2. Suitable SDLC Model

The **Agile Model** is the most suitable SDLC model for developing this healthcare system.

Justification:

1. **Changing requirements:** Hospital requirements may change as new departments, services, or features are introduced.
2. **Incremental development:** Features such as appointment booking, medical records, payments, and billing can be developed in separate iterations.
3. **Continuous testing:** Each module can be tested regularly to reduce errors.
4. **User feedback:** Doctors, patients, and administrators can provide feedback after every iteration.
5. **Security:** Sensitive medical and payment data can be reviewed and secured continuously.
6. **Early delivery:** Important features such as appointment booking and medical records can be released before the complete system is finished.

Smart Healthcare System — User Inheritance Hierarchy



MedicalRecordId is private to Patient and only exposed through controlled methods (encapsulation), protecting sensitive health data while getDashboard() is polymorphic per role.

Conclusion

Using **encapsulation, inheritance, and polymorphism** makes the healthcare system secure, reusable, and maintainable. **Agile SDLC** is recommended because it supports continuous testing, user feedback, changing requirements, and incremental delivery, which are important for a complex healthcare system.

6. Smart Library Management System

A university library intends to automate its operations, allowing students and faculty members to search books, borrow and return books, reserve titles online, and pay overdue fines. Librarians manage book inventories, memberships, and circulation records, while administrators generate reports and maintain the system.

Question:

Design a structured solution using **object-oriented concepts** for this system. Identify the major classes, attributes, methods, and relationships, and recommend an appropriate **SDLC model** for efficient development with suitable justification.

ANSWER :

1. Object-Oriented Design

The Smart Library Management System can be designed using object-oriented concepts by dividing the system into different classes.

2. Major Classes, Attributes and Methods

Class	Main Attributes	Main Methods
User	userID, name, email, phone	login(), logout()
Student	studentID, course	borrowBook(), returnBook(), reserveBook()
Faculty	facultyID, department	borrowBook(), returnBook(), reserveBook()
Book	bookID, title, author, ISBN, status	searchBook(), checkAvailability()
Librarian	librarianID, name	addBook(), removeBook(), updateBook()
Library	libraryID, name	searchBook(), issueBook(), receiveBook()
Reservation	reservationID, date, status	createReservation(), cancelReservation()
Fine	fineID, amount, dueDate, status	calculateFine(), payFine()
Administrator	adminID, name	generateReport(), manageSystem()

3. Relationships

- **Inheritance:** Student and Faculty inherit common properties from the User class.
- **Association:** A User can borrow multiple Book objects.

- **Association:** A User can create a Reservation.
- **Composition:** A Library contains books, librarians, and circulation records.
- **Association:** A Fine is associated with a user when a book is returned late.
- **Dependency:** Administrator uses library data to generate reports.

4. OOP Concepts Used

- **Encapsulation:** Book details, membership information, and fine details are protected inside their classes.
- **Inheritance:** Student and Faculty classes reuse common User properties and methods.
- **Polymorphism:** borrowBook() can have different borrowing rules for students and faculty members.
- **Abstraction:** Users can perform operations such as searchBook() without knowing the internal database implementation.

5. Suitable SDLC Model

The Agile Model is suitable for developing the Smart Library Management System.

Justification:

1. Requirements can change based on student, faculty, and librarian feedback.
2. The system can be developed incrementally into modules such as search, borrowing, reservation, fines, and reporting.
3. Continuous testing helps identify errors early.
4. New features such as mobile access, digital books, or notifications can be added easily.
5. Regular feedback from librarians and users improves usability.
6. Working modules can be delivered quickly instead of waiting for the entire system.

Conclusion

The system can be effectively designed using OOP concepts such as encapsulation, inheritance, polymorphism, abstraction, and relationships between classes. Agile SDLC is recommended because it supports modular development, continuous testing, user feedback, and changing requirements, making the library system efficient and maintainable.

RUBRICS FOR CALCULATION:

Case study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand